

Mobile Robotics Lab Manual

Lab Session 4: Introduction to ROS 2 Launch, Rosbag, rqt-plot

1. Objective

- To understand and utilize the ROS 2 launch system.
- To record and replay ROS 2 topics using Rosbag.
- To visualize and analyze topic data using rqt.

2. Prerequisites

Before starting this lab, students should:

- Have ROS 2 Humble installed on their systems.
- Be familiar with ROS 2 basic concepts such as nodes, topics, and services.
- Have completed Lab Sessions (Turtlesim and ROS 2 Basics).

3. Lab Setup

1. Open a terminal and source your ROS 2 environment:

```
source /opt/ros/humble/setup.bash
```

2. Ensure the necessary packages are installed:

```
sudo apt install ros-humble-turtlesim ros-humble-roscpp
```

4. Step-by-Step Instructions

Step 1: Understanding ROS 2 Launch Files

A ROS 2 launch file is used to start multiple nodes simultaneously with predefined configurations. This is especially useful for complex robotics applications where several nodes need to be started together.

Creating a ROS 2 Launch File

1. Navigate to your ROS 2 workspace:

```
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws/src
```

2. Create a new package for launch files:

```
ros2 pkg create my_launch_pkg --build-type ament_python --
dependencies turtlesim
```

3. Create a launch directory inside the package:

```
mkdir -p my_launch_pkg/launch
```

In order for colcon to find the launch files, we need to inform Python's setup tools of our launch files using the `data_files` parameter of `setup`.

Inside our `setup.py` file:

```
import os
from glob import glob
from setuptools import setup

package_name = 'py_launch_example'

setup(
    # Other parameters ...
    data_files=[
        # ... Other data files
        # Include all launch files.
        (os.path.join('share', package_name, 'launch'),
        glob(os.path.join('launch', '*launch.[pxy][yma]*'))
        ]
    )
```

4. Create a new launch file `turtlesim_launch.py` inside launch directory:

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='turtlesim',
            executable='turtle_teleop_key',
            name='teleop'
        )
    ])
```

5. Build the package:

```
cd ~/ros2_ws && colcon build
```

6. Source the workspace:

```
source install/setup.bash
```

7. Launch the simulation:

```
ros2 launch my_launch_pkg turtlesim_launch.py
```

Step 2: Understanding Rosbag

Rosbag is a tool in ROS 2 used for recording and replaying messages passed between nodes. It allows users to capture data for later analysis and debugging.

Recording and Playing Data with Rosbag

1. Start a ROS 2 bag recording:

```
ros2 bag record /turtle1/pose
```

2. Move the turtle using keyboard teleoperation:

```
ros2 run turtlesim turtle_teleop_key
```

3. Stop recording with `Ctrl+C`.
4. Play back the recorded data:

```
ros2 bag play <bag_filename>
```

5. Observe the turtle moving based on the recorded data.

Step 3: Visualizing Data with rqt

ROS 2 provides `rqt`, a graphical user interface (GUI) tool to visualize topics and other ROS elements.

Plotting `/turtle1/cmd_vel` in rqt

1. Ensure `rqt` is installed:

```
sudo apt install ros-humble-rqt ros-humble-rqt-graph ros-humble-rqt-plot
```

2. Run `rqt`:

```
rqt
```

3. Open the `rqt_plot` plugin by selecting `Plugins` → `Visualization` → `Plot`.
4. In the topic field, enter `/turtle1/cmd_vel` and observe the velocity changes when controlling the turtle.

5. Lab Tasks

1. Modify the launch file to include multiple nodes (e.g., a second turtle).
2. Record additional topics such as `/turtle1/cmd_vel` and `/turtle2/cmd_vel`.
3. **Implement a Follow-the-Leader Behavior:** Create a ROS 2 node where a second turtle follows the first turtle by subscribing to the first turtle's pose and publishing appropriate velocity commands to the second turtle.

4. **Use rqt to Plot Control Commands:** Use `rqt_plot` to visualize the `/turtle1/cmd_vel` topic while moving the turtle.

6. Deliverables

- Submit screenshots of the launch process and bag recording.
- Provide a short report detailing your approach, findings, and observations.
- Submit the modified launch file.
- Submit extracted trajectory data from the rosbag file with a brief analysis.
- Submit the implementation code for the follow-the-leader behavior.
- Submit the plotted graph of `/turtle1/cmd_vel` in `rqt_plot`.